

Task 1 of the Quantum Virtual Internship

load required libraries:

Pandas, re

Import pandas as pd

Import re

filePath <- "C:/Users/mm osman/Desktop/quantum_intern"

transactionData -> data = pd.read_csv("QVI_transaction_data.csv", filePath)

customerData -> df = pd.read_csv("QVI_purchase_behaviour.csv", filePath)

Exploratory data analysis

df.head()

df["PREMIUM_CUSTOMER"].unique()

data.head()

data.isnull().sum()

df.isnull().sum()

print(df.columns.to_list())

print(data.columns.to_list())

data.dtypes “and check each data type if relative for each column”

df.dtypes “and check each data type if relative for each column”

➔ But you will see that the DATE column is integer and for this we want to convert it to Date_time type

➔ Convert int type for date to Date_time

```
➔ data["DATE"] = pd.to_datetime(data['DATE'], origin='1899-12-30', unit='D')
```

TO know that all the product names are chips we will split all the product names into words and count them and try to examine that if these products words may be in the product names or not:

```
from collections import Counter
```

```
# Split all product names into individual words
```

```
all_words = ' '.join(df_merged['PROD_NAME'].unique()).split()
```

```
# Count each word
```

```
word_counts = Counter(all_words)
```

```
# Convert to dataframe and sort
```

```
word_df = pd.DataFrame(word_counts.items(), columns=['Word',  
'Count']).sort_values('Count', ascending=False)
```

```
print(word_df.head(20))
```

To see if those words are right, we will show those entities that suspicious words are appeared like salsa and Dip words,

```
df_merged[df_merged['PROD_NAME'].str.contains('Salsa',  
case=False)][['PROD_NAME'].unique()]
```

we observed that some of the outputs are right a chip and some are not a chip but a dip, for this we will drop those outputs

```
non_chips = [
```

```
'Old El Paso Salsa Dip Tomato Med 300g',
```

```
'Old El Paso Salsa Dip Tomato Mild 300g',  
'Old El Paso Salsa Dip Chnky Tom Ht300g',  
'Woolworths Medium Salsa 300g',  
'Woolworths Mild Salsa 300g'  
]
```

```
df_merged = df_merged[~df_merged['PROD_NAME'].isin(non_chips)]
```

but there's a challenging part and it's that we want to only keep on only useful words that can enable us to specify whether this word is a chip or not and this is the code of it:

```
import re
```

```
# Split all product names into individual words
```

```
all_words = ' '.join(df_merged['PROD_NAME'].unique()).split()
```

```
# Remove words with digits and special characters
```

```
clean_words = [word for word in all_words if re.match(r'^[A-Za-z]+$', word)]
```

```
# Count and sort
```

```
word_counts = Counter(clean_words)
```

```
word_df = pd.DataFrame(word_counts.items(), columns=['Word',  
'Count']).sort_values('Count', ascending=False)
```

```
print(word_df.head(20))
```

To see the missing data in the date column, we founded that the date that missing, it's the Christmas date as all supermarkets are closed in this day:

```
df_merged.groupby('DATE')['TXN_ID'].count().reset_index().sort_values('DATE')
```

and to see which date is missing, it's

```
# Create a full date range
```

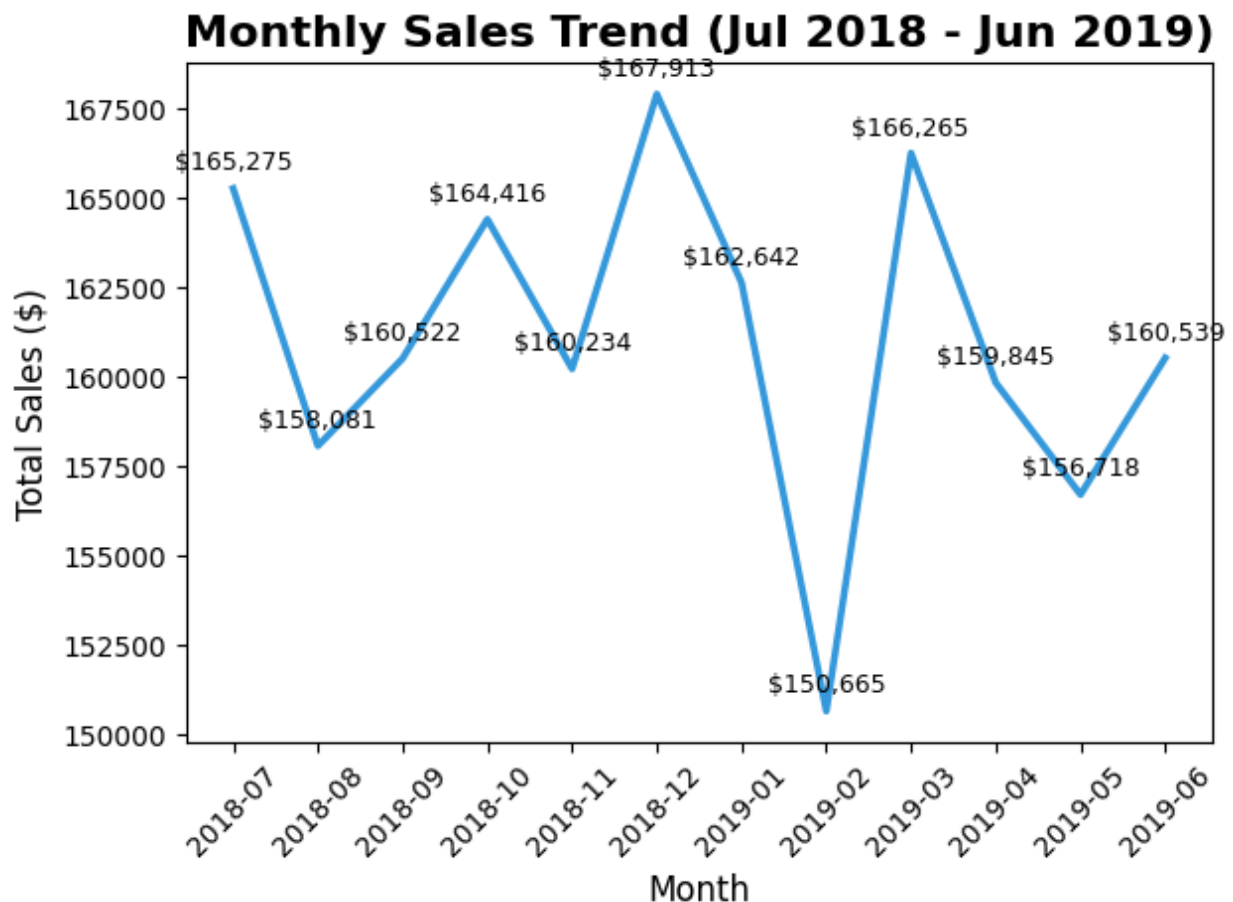
```
full_dates = pd.date_range(start='2018-07-01', end='2019-06-30')
```

```
# Find missing dates
```

```
missing_dates = full_dates[~full_dates.isin(df_merged['DATE'])]
```

```
print(missing_dates)
```

We can see that there is an increase in purchases in December and a break in late December. Let's zoom in on this



We can see that the increase in sales occurs in the lead-up to Christmas and that there are zero sales on Christmas day itself. This is due to shops being closed on Christmas day.

Now that we are satisfied that the data no longer has outliers, we can move on to creating other features such as brand of chips or pack size from PROD_NAME. We will start with pack size

create brand column:

And creating packet size column also:

```
data["brand"] = data["PROD_NAME"].str.split().str[0]

data["packet_size"] = data["PROD_NAME"].str.extract(r'(\d+)g',
flags=re.IGNORECASE)
```

The largest size is 380g and the smallest size is 70g - seems sensible!

Let's plot a histogram of PACK_SIZE since we know that it is a categorical variable and not a continuous variable even though it is numeric.

And this is the code for this plotting:

```
packet_size_data2 =
df_merged.groupby('packet_size')['LYLTY_CARD_NBR'].nunique().reset_index()

packet_size_data2.columns = ['packet_size', 'NUM_CUSTOMERS']
```

```
plt.figure(figsize=(14, 7))
```

```
sns.barplot(
    data=packet_size_data2,
    x='packet_size',
    y='NUM_CUSTOMERS',
    palette='Set2'
)
```

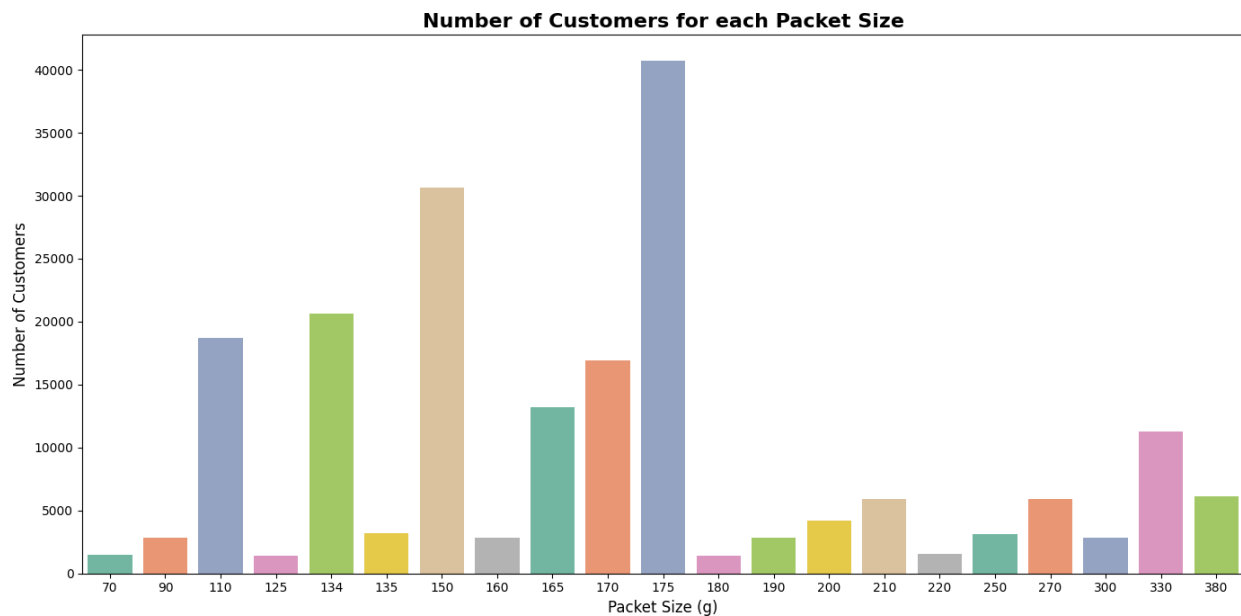
```
plt.title('Number of Customers for each Packet Size', fontsize=16, fontweight='bold')
```

```
plt.xlabel('Packet Size (g)', fontsize=12)
```

```
plt.ylabel('Number of Customers', fontsize=12)
```

```
plt.tight_layout()
```

```
plt.show()
```



FOR THE ADJUSTMENTS of the brand,

Yes, there's a lot of brands have the same name but in different ways may be have less characters or an abbreviation for the brand. For the adjustment of it, we will add this code:

```
clean_brand = {  
  
    'Smith': 'Smiths',  
  
    'Snbts': 'Sunbites',  
  
    'Infzns': 'Infuzions',  
  
    'Dorito': 'Doritos',  
  
    'GrnWves': 'Grain',  
  
    'NCC': 'Natural',  
  
    'RRD': 'Red',
```

```
'WW': 'Woolworths'
}

df_merged["brand"] = df_merged["brand"].replace(clean_brand)
```

Data analysis on customer segments

Let's start with calculating total sales by LIFESTAGE and PREMIUM_CUSTOMER and plotting the split by these segments to describe which customer segment contribute most to chip sales.

Using this code:

```
segment_sales = df_merged.groupby(["LIFESTAGE",
"PREMIUM_CUSTOMER"])[["TOT_SALES"].sum().reset_index().sort_values("TOT_SALES"
, ascending= False)

plt.figure(figsize=(14, 7))

sns.barplot(data=segment_sales, x="TOT_SALES", y="LIFESTAGE",
hue="PREMIUM_CUSTOMER", palette={'Budget': '#2ecc71', 'Mainstream': '#3498db',
'Premium': '#e74c3c'})

plt.title("Total sales by customer segment", fontsize=16, fontweight='bold')

plt.xlabel('Total_sales($)', fontsize=12)

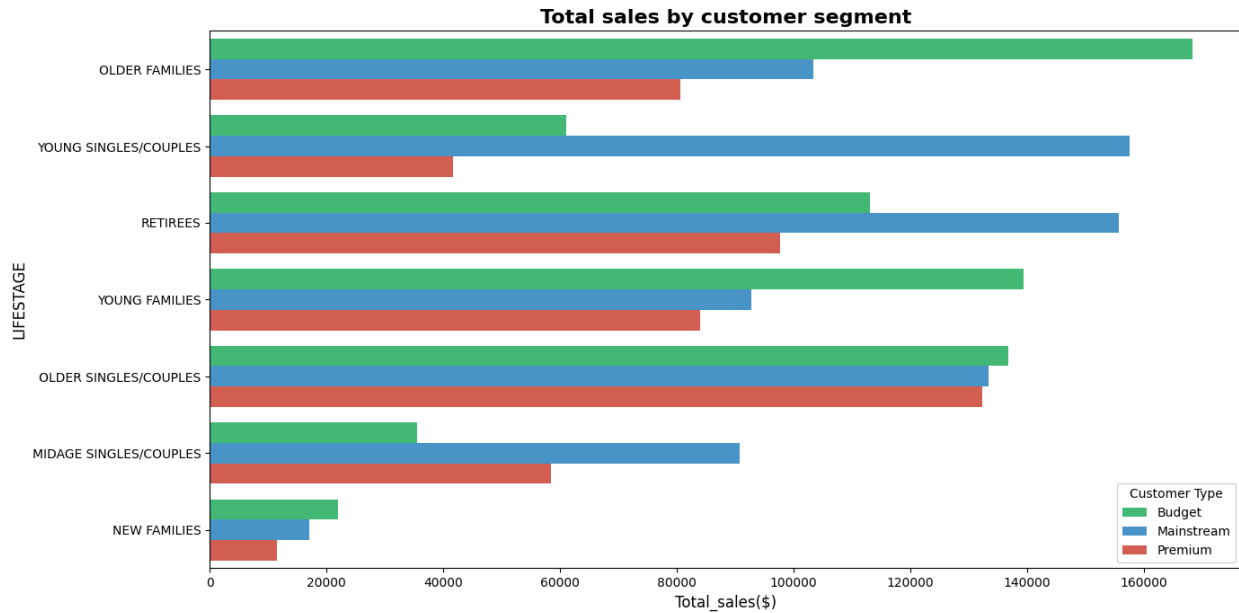
plt.ylabel("LIFESTAGE", fontsize=12)

plt.legend(title= 'Customer Type')

plt.tight_layout()

plt.show()
```

and this is the visualization of the highest segments by TOT_SALES



We now see that the top 3 highest segments are **(OLDER FAMILIES – Budget)-> (YOUNG SINGLES/COUPLES-Mainstream) -> (RETIREEES-Mainstream)**

But we want to see now what is the driver for spendings for these chips, now we want to see if the **number of customers** is the driver for these spending or not!!!

The code of number of customers:

```
segment_customer_number = df_merged.groupby(['LIFESTAGE',
'PREMIUM_CUSTOMER'])['LYLTY_CARD_NBR'].nunique().reset_index()
```

```
segment_customer_number.columns = ['LIFESTAGE', 'PREMIUM_CUSTOMER',
'NUM_CUSTOMERS']
```

```
plt.figure(figsize=(14, 7))
```

```
sns.barplot(
```

```
    data=segment_customer_number,
```

```
    x='NUM_CUSTOMERS',
```

```
    y='LIFESTAGE',
```

```
    hue='PREMIUM_CUSTOMER',
```

```
    palette='Set1'
```


)

```
plt.title("Number of Customers in each Segment", fontsize=16, fontweight='bold')
```

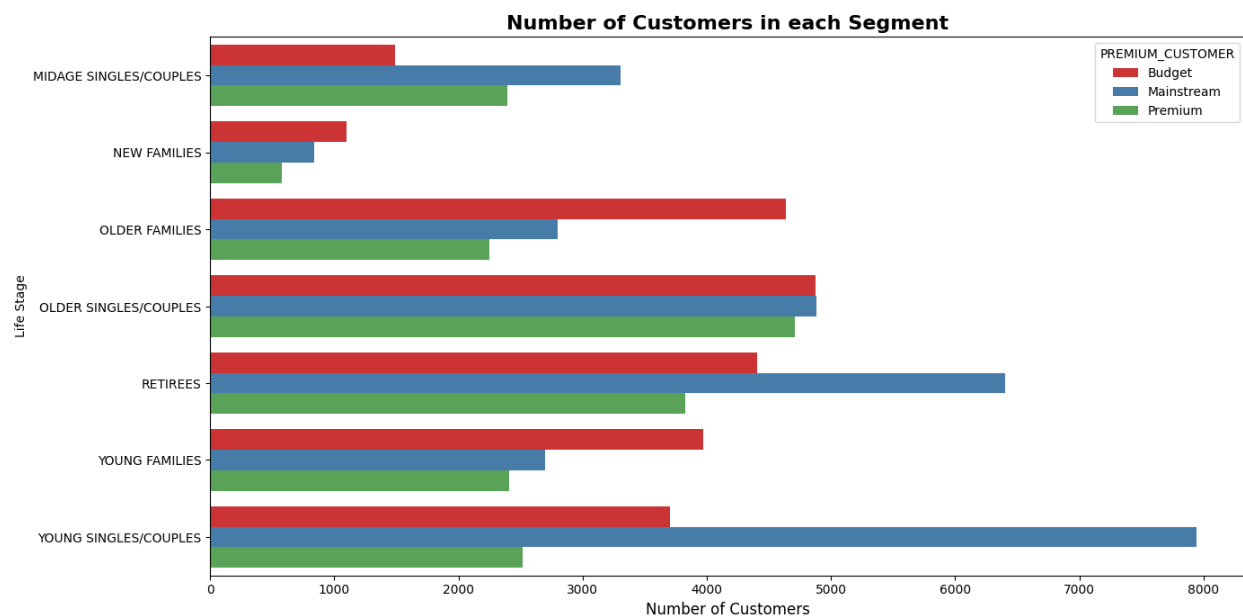
```
plt.xlabel('Number of Customers', fontsize=12)
```

```
plt.ylabel('Life Stage')
```

```
plt.tight_layout()
```

```
plt.show()
```

and this is the plot



We observed here that the number of customers is a driver for the two segments of **(YOUNG SINGLES/COUPLES – Mainstream)** and **(RETIREES-Mainstream)** , but not for the segment of **(OLDER FAMILIES-Budget)**.

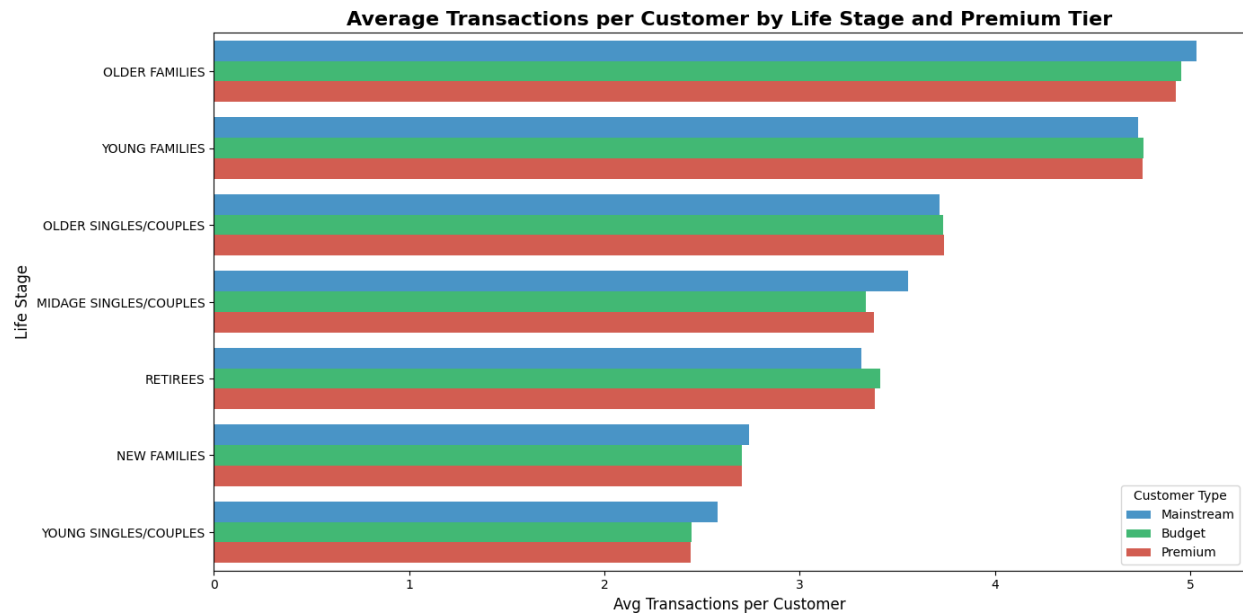
Let's see if the **number of units** been bought by each customer may be a driver for spendings for these chips:

We observed that **(OLDER FAMILIES and YOUNG FAMILIES)** are buying more and that's actually very clear as families buy more units as the number of families as big than singles.

This is the code of making this plot:

```
num_transaction_per_customer = df_merged.groupby(['LIFESTAGE',  
'PREMIUM_CUSTOMER']).agg(  
  
    AVG_TRANSACTIONS=('TXN_ID', 'count'),  
  
    NUM_CUSTOMERS=('LYLTY_CARD_NBR', 'nunique')  
)  
.assign(AVG_TXN_PER_CUSTOMER=lambda x: x['AVG_TRANSACTIONS'] /  
x['NUM_CUSTOMERS']).reset_index().sort_values('AVG_TXN_PER_CUSTOMER',  
ascending=False)
```

```
plt.figure(figsize=(14, 7))  
  
sns.barplot(  
  
    data=num_transaction_per_customer,  
  
    x='AVG_TXN_PER_CUSTOMER',  
  
    y='LIFESTAGE',  
  
    hue='PREMIUM_CUSTOMER',  
  
    palette={'Budget': '#2ecc71', 'Mainstream': '#3498db', 'Premium': '#e74c3c'}  
)  
  
plt.title('Average Transactions per Customer by Life Stage and Premium Tier',  
fontsize=16, fontweight='bold')  
  
plt.xlabel('Avg Transactions per Customer', fontsize=12)  
  
plt.ylabel('Life Stage', fontsize=12)  
  
plt.legend(title='Customer Type')  
  
plt.tight_layout()  
  
plt.show()
```



Here we observe that one of the drivers of the remaining segment of OLDER FAMILIES is the number of units been bought by each customer.

And now we also must see if the price of each unit bought by each customer is a driver for spending or not:

THIS IS THE CODE FOR AVERAGE PRICE FOR EACH UNIT in the segment:

```
avg_price_per_unit_sold = df_merged.groupby(['LIFESTAGE',
'PREMIUM_CUSTOMER']).agg(
    AVG_TRANSACTIONS=('TXN_ID', 'count'),
    NUM_CUSTOMERS=('LYLTY_CARD_NBR', 'nunique'),
    TOT_SALES=('TOT_SALES', 'sum'),
    TOT_UNITS=('PROD_QTY', 'sum')
).assign(
    AVG_PRICE_PER_UNIT=lambda x: x['TOT_SALES'] / x['TOT_UNITS']
).reset_index().sort_values('AVG_PRICE_PER_UNIT', ascending=False)

plt.figure(figsize=(14, 7))
sns.barplot(
```

```

data=avg_price_per_unit_sold,
x='AVG_PRICE_PER_UNIT',
y='LIFESTAGE',
hue='PREMIUM_CUSTOMER',
palette={'Budget': '#2ecc71', 'Mainstream': '#3498db', 'Premium': '#e74c3c'}
)

plt.title("Average Price per Unit Sold", fontsize=16, fontweight='bold')
plt.xlabel('Average Price per Unit ($)', fontsize=12)
plt.ylabel("Life Stage", fontsize=12)
plt.legend(title='Customer Type')
plt.tight_layout()
plt.show()

```



We here observe that the highest segment is (YOUNG SINGLES/COUPLES- Mainstream) and (MIDAGE SINGLES/COUPLES- Mainstream), But what I see here is that the Average price per unit is very close to each other as the range is between (3.70\$-4.0\$)

Now here we want to see if there's a difference between the segment of Mainstream vs budget and premium:

```
mainstream4 = df_merged[(df_merged["PREMIUM_CUSTOMER"]=="Mainstream")&
(df_merged["LIFESTAGE"].isin(['YOUNG SINGLES/COUPLES', 'MIDAGE
SINGLES/COUPLES']))
][['UNIT_PRICE']]
```

```
budget_premium4 = df_merged[(df_merged["PREMIUM_CUSTOMER"].isin(["Premium",
"Budget"]))&(df_merged["LIFESTAGE"].isin(['YOUNG SINGLES/COUPLES', 'MIDAGE
SINGLES/COUPLES']))][['UNIT_PRICE']]
```

```
t_stat4, p_value4 = stats.ttest_ind(mainstream4, budget_premium4)
```

```
print(f"Mainstream VS Budget_Premium → t-stat: {t_stat4:.4f}, p-value:{p_value4:.4f}")
```

The t-test results in a p-value of 0.0000, and the t-stat is 38.7734, i.e. the unit price for mainstream, young and mid-age singles and couples **[ARE / ARE NOT]** significantly higher than that of budget or premium, young and midage singles and couples.

What we see here is that **Mainstream is spending more than budget and premium** and that's for UNIT_PRICE and the **Premium is spending slightly more than budget segment**

For this we can focus more for advertising for Mainstream MIDAGE and YOUNG SINGLES and COUPLES

For example for this segment of Mainstream YOUNG SINGLES & COUPLES: we want to see what is the best preference brand for them:

Total transactions per brand for everyone

```
total_brand = df_merged.groupby('brand')['TXN_ID'].count() / len(df_merged)
```

```
# Transactions per brand for Mainstream Young Singles/Couples only
```

```
target_segment = df_merged[  
    (df_merged['PREMIUM_CUSTOMER'] == 'Mainstream') &  
    (df_merged['LIFESTAGE'] == 'YOUNG SINGLES/COUPLES')  
]
```

```
segment_brand = target_segment.groupby('brand')['TXN_ID'].count() /  
len(target_segment)
```

```
# Calculate affinity
```

```
affinity = (segment_brand / total_brand).reset_index()  
affinity.columns = ['brand', 'AFFINITY']  
affinity = affinity.sort_values('AFFINITY', ascending=False)
```

```
print(affinity)
```

Key Insights 🎯

1. They prefer premium/fun brands

- Tyrrells, Twisties, Tostitos, Pringles are all trendy and fun brands
- Makes sense for Young Singles who buy chips for social occasions and parties

2. They avoid supermarket/health brands

- Woolworths (generic supermarket brand) → strongly avoided
- Sunbites (healthy snack) → strongly avoided
- Young Singles don't want boring or healthy chips!

3. Kettle is still strong

- Confirms our earlier finding that Kettle is universally popular

Business Recommendation for Julia 🎯

For Mainstream Young Singles/Couples:

- Increase shelf space for Tyrrells, Twisties, Tostitos, Pringles
- Run promotions on Kettle and Doritos
- Don't waste marketing budget on Woolworths or Sunbites for this segment

And for packet_size preference I found that using the same method for the preference of the brand

Total transactions per brand for everyone

```
total_packet_size = df_merged.groupby('packet_size')['TXN_ID'].count() /
len(df_merged)
```

Transactions per brand for Mainstream Young Singles/Couples only

```
target_segment = df_merged[
    (df_merged['PREMIUM_CUSTOMER'] == 'Mainstream') &
    (df_merged['LIFESTAGE'] == 'YOUNG SINGLES/COUPLES')
]
```

```
segment_packet_size = target_segment.groupby('packet_size')['TXN_ID'].count() /
len(target_segment)
```

Calculate affinity

```
affinity = (segment_packet_size / total_packet_size).reset_index()
affinity.columns = ['packet_size', 'AFFINITY']
affinity = affinity.sort_values('AFFINITY', ascending=False)
```

print(affinity)

I founded that the

Key Insights 

1. They prefer LARGER pack sizes

- 270g, 380g, 330g are all at the top
- Makes sense for Young Singles buying chips for parties and social gatherings — they need more!

2. They also like SMALLER packs

- 110g, 134g appear in top preferences
- Could be for individual snacking on the go

3. They avoid MEDIUM sizes

- 160g, 180g, 190g, 200g are all strongly avoided
- Interesting — they go for either big or small, nothing in between!

And this is a complete for Julia for the segment of Mainstream-MIDAGE/YOUNG SINGLES AND COUPLES

Mainstream Young Singles/Couples want: → Trendy brands : Tyrrells, Twisties, Tostitos, Pringles → Either large packs (270g-380g) for parties → Or small packs (110g-134g) for on-the-go snacking → NOT medium sizes (160g-200g) → NOT generic brands (Woolworths, Sunbites)